

Metody pozycyjne sortowania

wykorzystują wartość klucza (liczba, znak, sznur znaków itp.)

1. Countsort: sortowanie metodą zliczania.

Sortujemy tablicę $A[0..n-1]$, $A[i] \in 0..k-1$, $i=0, \dots, n-1$.

Tablice robocze:

$B[0..n-1]$, wynik (ciąg posortowany)

$C[0..k-1]$, zliczanie wartości, ustalenie baz

```
1. for j=0 to k-1 do           // inicjalizacja
    C[j] := 0;

2. for i=0 to n-1 do           // zliczanie
    C[A[i]]++;
                                // teraz C[j] = #{i: A[i]=j}

3. for j=1 to k-1 do           // obliczanie górnych baz
    C[j] := C[j-1] + C[j];
                                // teraz C[j] = #{i: A[i]≤j}

4. for i=n-1 downto 0 do       // ustal wynik
    B[--C[A[i]]] := A[i];
```

Przykład:

Przykład: $n=10$, $k=5$

wartości:	2	1	2	0	4	0	2	2	4	0
indeksy:	0	1	2	3	4	5	6	7	8	9

Liczba wystąpień poszczególnych wartości:

$C[0] = 3$, $C[1] = 1$, $C[2] = 4$, $C[3] = 0$, $C[4] = 2$

górne bazy (czyli sumy prefiksowe liczby wystąpień)

$C[0] = 3$, $C[1] = 4$, $C[2] = 8$, $C[3] = 8$, $C[4] = 10$

Uwagi do konstrukcji Countsort-u:

- obliczamy górne bazy a nie dolne po to, by móc umieścić je w tej samej tablicy $C[]$, którą użyliśmy do zliczania;
- ponieważ mamy górne bazy, przepisywanie do tablicy wynikowej, aby zachować stabilność, musi odbyć się po malejących indeksach;
- aby utrzymać aktualny adres wynikowy dla elementów o wartości j , przy każdym wpisaniu elementu $A[i]=j$ do wyniku, zmniejszamy $C[j]$, co powoduje, że kolejny element o wartości j zostanie wpisany pod adresem o 1 mniejszym.

Złożoność: $\Theta(n+k)$.

Pętle 1, 3: $O(k)$, pętle 2, 4: $O(n)$.

Zalety Countsort-u:

1. Jeśli $k=O(n)$ to algorytm ma złożoność liniową, $\Theta(n)$.
2. Stabilność.

Wady:

1. Niezbędna pamięć robocza wielkości $\Theta(n+k)$
2. Z powodu silnej zależności efektywności od postaci klucza nieczęsto nadaje się do wykorzystania.

Ostatnią wadę częściowo eliminuje Radixsort.

2. Radixsort (sortowanie pozycyjne)

Sytuacja:

klucze długie lub duże, tzn. ich zakres wartości (parametr k) zbyt duży do zastosowania Countsort-u, albo klucze złożone z kilku składowych.

Pomysł:

- podzielić klucze na kilka części, np. cyfry (dziesiętne, bajty, znaki, itp.);
- zastosować Countsort tyle razy ile jest części, każdą z nich traktując jako klucz w jednym przebiegu;
- wywołania Countsort-u dla poszczególnych części kluczy wykonywać w kolejności od najmniej do najbardziej znaczących;
- kluczową własnością Countsortu dla poprawności Radixsortu jest jego stabilność;
- zamiast Countsortu można zastosować dowolny algorytm sortowania stabilnego;
- Radixsort jest stabilny.

Przykład:

102 220 120 012 210 011 100 121 021 202 022 200 201

po sortowaniu względem ostatniej cyfry:

220 120 210 100 200 011 121 021 201 102 012 202 022

otrzymany wynik sortuj względem środkowej cyfry:

100 200 201 102 202 210 011 012 220 120 121 021 022

a ten wynik względem cyfry najbardziej znaczącej:

011 012 021 022 100 102 120 121 200 201 202 210 220

Przykładowa konstrukcja:

- elementy tablicy A są b-bajtowe; bajt zerowy jest najmniej znaczący;
- procedura `CountsortByByte(A,j)` sortuje tablicę A metodą zliczania traktując jako klucz j-ty bajt każdego elementu, $k=256$.

```
SampleRadixsort(A, b);  
    for j=0 to b-1 do CountsortByByte(A, j);  
end.
```

Kładąc $k=256=O(1)$ otrzymujemy:

- złożoność czasową: $\Theta(b*(n+k)) = \Theta(b*n)$,
- oraz pamięciową: $\Theta(n+k) = \Theta(n)$.

3. Bucketsort (sortowanie przez rozrzucanie) (sortowanie kubełkowe)

Podobnie jak w Countsort: $A[0..n-1]$, $A[i] \in 0..k-1$,
Zakładamy $k=O(n)$, ale praktycznie najczęściej $k=O(1)$.

```
1. for j=0 to k-1 do  
    CreateQueue Q[j];                                // "buckets"  
2. for i=0 to n-1 do  
    Q[A[i]].Insert(A[i]);                             // distribute  
3. i := 0;  
4. for j=0 to k-1 do    // concatenate queues into A  
    while Q[j] ≠ ∅ do  
        A[i++] := Q[j].Delete();
```

Złożoność czasowa i pamięciowa: $\Theta(n+k)$.

Stabilny, bo z kolejki usuwamy w kolejności wstawiania.

4. Multiplelistsort 1

(sort za pomocą wielu list, nazwa niejednoznaczna)

Uogólnienie Bucketsort-u, podobnie jak Radixsort jest uogólnieniem Countsort-u:

- klucze podzielone na b składowych (cyfr),
- cyfra ma wartość z zakresu $0..k-1$,
- iterujemy po pozycjach kolejnych cyfr od najmniej znaczącej,
- dla ustalonej pozycji j wykonujemy Bucketsort, gdzie kluczami są cyfry na tej pozycji,
- j -tą cyfrę klucza $A[i]$ oznaczamy przez $A[i][j]$.

```
MultipleListSort(A, b);  
// zał. k=10  
// L[0],...,L[9] - listy początkowo puste  
  
1. for j=0 to b-1 do  
2.   for i=0 to n-1 do  
3.     wstaw A[i] na koniec listy L[A[i][j]];  
  
4.   A[0..n-1] := L[0]^L[1]^...^L[9];  
                                     // konkatencja  
5.   L[0]:=L[1]:=...:=L[9]:= ∅ // lista pusta  
end.
```

Metoda kubełkowa stosowana była do mechaniczno-optycznego sortowania kart perforowanych (karty Holleritha).

5. Sortowanie za pomocą wielu list z wykorzystaniem sortowania przez wstawianie

- klucze są liczbami z przedziału $[c, d)$;
- dzielimy przedział $[c, d)$ na m podprzedziałów jednakowej długości $h=(d-c)/m$;
- dla każdego podprzedziału tworzymy listę początkowo pustą,
- lista ta będzie zawierać posortowane klucze o wartościach w tym podprzedziale;
- wykonujemy poniższą iterację:

```
for i=1 to n do
  j := numer podprzedziału do którego należy a[i];
  wstaw a[i] na właściwe miejsce w liście j;
  // metodą prostego wstawiania
```

- łączymy listy (konkatenacja) w kolejności przedziałów od lewej do prawej; KONIEC

Złożoność pesymistyczna:

$\Theta(n^2)$ – wszystkie elementy wpadają w jeden przedział.

Złożoność średnia:

Zakładamy rozkład równomierny kluczy, czyli:

- dla ustalonej listy prawdopodobieństwo, że klucz do niej wpada jest równe $1/m$ (schemat Bernoulli'ego).

Można wykazać (nietrudna kombinatoryka), że:

- oczekiwana liczba porównań wynosi $\Theta(n^2/m)$,
a zatem dobierając liczbę list m możemy osiągać złożoności z zakresu od $O(n)$ do $O(n^2)$.